

L21 — Vector Databases · guida agli script

Lab del sabato (📄 + 🎤 + 📊, 4h): la ricerca di L20 **trasloca in un vector DB**. Col docente si apre la collection ChromaDB su disco e si fa il quick start (`apri_collection` , i primi 3 doc, una query); poi gli studenti caricano DA SOLI il corpus completo (`carica_corpus_nel_db`), riscrivono la ricerca sopra il DB (`search`) e la confrontano fianco a fianco col proprio brute-force di giovedì; nel Lab 2 arrivano `search_filtrata` (metadata) e le prove CRUD (delete + update fatto bene). Chiude la demo pgvector del docente (materiale fuori da `scripts/`) e l'ultimo blocco flessibile: criteri di scelta, poi Q&A o l'esercizio extra "il 49° documento" (a scelta del docente). Niente notebook, solo script `.py` .

Il `check_lab21.py` è **strumento del docente**, fuori dal percorso studente (il "fatto quando" degli esercizi è l'output osservabile): serve per la preparazione/regressione dei materiali e per la diagnosi rapida di uno studente bloccato. Verifica i risultati su un DB usa-e-getta con query prese dai vettori GIÀ calcolati del corpus (istantaneo, zero costi, zero rete); il giro reale via API sta dietro `--live` .

Ruolo di ogni file

Legenda: 📦 ambiente/toolkit riusabile · 🎤 codice di riferimento che il docente live-coda · 📄 da completare dagli studenti · 📑 documenti/dati di esempio.

File	Ruolo	Cosa fa
<code>requirements.txt</code>	📦	Dipendenze (<code>chromadb</code> , <code>openai</code> , <code>numpy</code> ; <code>sentence-transformers</code> è commentato — si scommenta solo se il corpus di L19 è col backend <code>local</code>).
<code>.env.example</code>	📦	Template per <code>OPENAI_API_KEY</code> — la stessa chiave di L13-L20 , nessuna registrazione nuova.
<code>CONSEGNA.md</code>	📦	Promemoria operativo della giornata per gli studenti.
<code>scaffolding/aikit/embeddings.py</code>	📦	Il modulo di L19: <code>embed()</code> con backend selezionabile e <code>cosine_similarity()</code> . Non si tocca : si importa.
<code>scaffolding/aikit/search.py</code>	📦	Il brute-force scritto dagli studenti a L20 , promosso nel toolkit: stamattina è il termine di confronto per la ricerca sul DB. Non si tocca : si importa.

File	Ruolo	Cosa fa
<code>dataset/corpus_lumen.jsonl</code>		I 48 testi brevi di Lumen S.r.l. (12 FAQ, 12 prodotti, 12 ticket, 12 recensioni): il campo <code>tipo</code> diventa il metadata su cui si filtra.
<code>dataset/corpus_embeddings.npz</code> + <code>corpus_meta.json</code>		Il corpus vettorizzato: copia di riserva identica a quella di L19–L20 — chi ha i propri file li copia qui sopra.
<code>working/vectorstore.py</code>		L'unico file della giornata. TODO 1 col docente (<code>apri_collection</code> + quick start nel main); TODO 2–4 da soli (<code>carica_corpus_nel_db</code> , <code>search</code> , <code>search_filtrata</code> — prescritti solo nomi, variabili del main e scopo); TODO 5 nel main (prove CRUD: delete + update); TODO 6 extra (il 49° documento, solo se il docente lo lancia). Riferimento in <code>solutions/</code> .
<code>scaffolding/check_lab21.py</code>		Strumento del docente (preparazione, regressione, diagnosi): <code>[OK]/[FAIL]</code> sui risultati, su un DB usa-e-getta — istantaneo, zero costi, zero rete; <code>--live</code> fa 2 query vere (~\$0.000001); <code>--solutions</code> verifica le soluzioni. Non compare nel percorso studente.
<code>scaffolding/benchmark_ann.py</code>		Brute-force vs indice ChromaDB sugli stessi vettori sintetici di L20: i numeri del “reveal” citati in slide.
<code>solutions/</code>		Le versioni complete dei soli file di working/ (stesso nome, stessi import).

La cartella `chroma/` (il DB su disco) nasce al primo lancio e NON si versiona (`.gitignore`); la demo pgvector del docente vive **fuori da scripts/**: `demo-pgvector-docente.md` + `demo-pgvector-carica.py` nella cartella della lezione.

Setup

```
cd scripts
python3 -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env # dentro: la stessa chiave OpenAI di L13-L20
```

Nota per chi distribuisce (docente): agli studenti la cartella arriva **senza** `.env`, `.venv` e `chroma/` (git non li versiona). Chi ha il proprio corpus di L19 lo copia in `dataset/` sopra la riserva; per tutti gli altri la riserva funziona identica.

Come si prova

Tutto si lancia dalla cartella `scripts/`:

```
python working/vectorstore.py # le prove della giornata (query via API,
    ~$0.000005 a giro completo)
python scaffolding/check_lab21.py # verifica working/: istantanea, zero costi
python scaffolding/check_lab21.py --live # 2 query vere via OpenAI (~$0.000001)
python scaffolding/check_lab21.py --solutions # per il docente: verifica solutions/
python scaffolding/benchmark_ann.py # brute-force vs indice, misurato (~40 s)
```

Le idee da trasmettere (coi numeri misurati, luglio 2026)

- 1. Il DB fa la stessa ricerca di giovedì — verificato, non promesso.** Sulle tre query del main, ChromaDB e il brute-force di L20 mettono in testa gli STESSI documenti con gli STESSI score (0.756 / 0.595 / 0.588 sulle rispettive prime posizioni): stessa cosine, stessi vettori. Quello che cambia non è il risultato: è chi fa il lavoro, e come scala.
- 2. L'indice resta piatto dove il brute-force cresce dritto.** Stessi vettori sintetici di L20 (1536 dim, float32), una query: a 5k vettori brute 5.9 ms vs Chroma 1.9 ms; a 50k **brute 152.5 ms vs Chroma 2.6 ms**. In cambio l'indice si PAGA alla scrittura: 1.3 s per indicizzare 5k, **37.5 s per 50k**. Scrivi una volta, interroghi per sempre. (In slide: tempi secchi, il benchmark non si nomina.)
- 3. Gli embedding si passano espliciti, sempre.** Chroma ha un embedder di default (MiniLM, 384 dim) che NON è quello del corpus (1536): mai lasciarglielo usare — upsert e query viaggiano con embedding calcolati da noi, col backend scritto in `corpus_meta.json`. È la regola di L20, lato database.
- 4. Il DB parla in distanze, noi in score.** `collection.query` restituisce `distances` (cosine distance): lo score di L20 è `1 - distanza`. Fascia di controllo già nota: fuori dominio (carbonara) resta a **0.196**, la query facile a **0.756** — le fasce di giovedì valgono ancora.
- 5. Il filtro cambia la domanda, non la query.** “problemi con il motore della scrivania” senza filtro: tic-02 / rec-04 / faq-06 (0.588 / 0.574 / 0.557). Con `tipo="faq"`: solo FAQ (0.557 / 0.538 /

0.483); con `tipo="recensione"`: solo recensioni. Stessa semantica, recinti diversi — e il recinto lo decidono i metadata caricati con l'upsert.

6. Update = ri-embed + upsert, stesso id. Il DB NON si accorge dei testi cambiati: cambiare il testo di `faq-02` (da spedizione a ritiro in negozio) senza ricalcolare l'embedding lascerebbe la ricerca sul vecchio significato. Fatto bene: `faq-02` esce dalla query "spedizione" e compare su "posso ritirare in negozio?" a **0.718**. Testo ed embedding li sincronizziamo NOI.

7. Tre famiglie, una scelta di contesto. ChromaDB: embedded, zero infrastruttura, perfetto per prototipi e corsi. pgvector: il Postgres che già conosci con un tipo in più — ricerca = `ORDER BY ⇔ LIMIT k`, filtro = `WHERE`, e le JOIN coi tuoi dati (demo docente: stessi risultati, 0.588 / 0.574 / 0.557). Pinecone: managed, niente da gestire, si paga a consumo. I concetti (collection, vettori, metadata, ANN) sono gli stessi ovunque: si impara una volta, si sceglie per contesto.

Percorso in aula (le slide sono la regia: ogni switch ha la sua slide)

- 1. Brief/teoria** (, 35'): da brute-force a indice (i numeri del punto 2), modello dati del vector DB, le tre famiglie, la regola degli embedding espliciti.
- 2. Quick start** (, 15'): `apri_collection` + primi 3 doc + query di prova col docente (TODO 1); fatto quando rilanciando lo script i doc sono ancora lì.
- 3. Lab 1** (, 75'): `carica_corpus_nel_db` e `search` da soli (TODO 2-3); fatto quando `count()` dice 48 (e resta 48 al rilancio) e le tre query del main danno gli stessi documenti in testa del brute-force, fianco a fianco.
- 4. ☕ Pausa** (15').
- 5. Lab 2** (, 60'): `search_filtrata` (TODO 4) e le prove CRUD nel main (TODO 5); fatto quando la stessa query cambia faccia col filtro, il doc cancellato sparisce e quello aggiornato cambia query di appartenenza.
- 6. Demo pgvector** (, 15'): solo docente, materiale fuori da `scripts/`.
- 7. Criteri di scelta + ultimo blocco** (, 25'): la tabella di scelta a slide, poi due strade a scelta del docente — Q&A aperto (anche sulle lezioni passate) oppure l'esercizio extra "il 49° documento" (TODO 6: una voce nuova del corpus scritta dallo studente, embed + upsert con id nuovo; riferimento in `solutions/`). Chiude il wrap-up col ponte a L22 (documenti lunghi → chunking).